

Nexus ERP

Com certeza. Aqui está a consolidação de toda a arquitetura do ****Nexus ERP**** em ordem lógica de implementação. Este guia serve como o seu ****Blueprint de Desenvolvimento****.

Fase 1: Fundação – Banco de Dados (PostgreSQL)

Este script cria toda a estrutura necessária para suportar os 100% da empresa com rigor contábil.

```
` ``sql
```

```
-- Habilitar UUID para Ids mais seguros
```

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
-- 1. NÚCLEO E PARCEIROS
```

```
CREATE TABLE companies (
```

```
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    Name VARCHAR(255) NOT NULL,
```

```
    Tax_id VARCHAR(20) UNIQUE NOT NULL, -- CNPJ
```

```
    Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
CREATE TABLE business_partners (
```

```
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    Company_id UUID REFERENCES companies(id),
```

```
    Name VARCHAR(255) NOT NULL,
```

```
    Tax_id VARCHAR(20) UNIQUE NOT NULL,
```

```
    Category VARCHAR(20) CHECK (category IN ('CUSTOMER', 'VENDOR', 'BOTH',  
'EMPLOYEE')),  
  
    Credit_limit DECIMAL(15,2) DEFAULT 0.00,  
  
    Is_active BOOLEAN DEFAULT TRUE  
);
```

-- 2. FINANCEIRO (ESTRUTURA SAP)

```
CREATE TABLE chart_of_accounts (  
    Id SERIAL PRIMARY KEY,  
    Company_id UUID REFERENCES companies(id),  
    Code VARCHAR(20) NOT NULL,  
    Name VARCHAR(100) NOT NULL,  
    Account_type VARCHAR(20) CHECK (account_type IN ('ASSET', 'LIABILITY',  
'EQUITY', 'REVENUE', 'EXPENSE')),  
    UNIQUE(company_id, code)  
);
```

```
CREATE TABLE general_ledger (  
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    Company_id UUID REFERENCES companies(id),  
    Transaction_date DATE DEFAULT CURRENT_DATE,  
    Description TEXT,  
    Reference_doc_id UUID,  
    Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE ledger_items (  
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    Ledger_id UUID REFERENCES general_ledger(id) ON DELETE CASCADE,
```

```
Account_id INTEGER REFERENCES chart_of_accounts(id),  
Debit DECIMAL(15,2) DEFAULT 0.00,  
Credit DECIMAL(15,2) DEFAULT 0.00,  
CHECK (debit >= 0 AND credit >= 0)  
);
```

-- 3. OPERACIONAL E PRODUTOS

```
CREATE TABLE products (  
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    Company_id UUID REFERENCES companies(id),  
    Sku VARCHAR(50) UNIQUE NOT NULL,  
    Name VARCHAR(255) NOT NULL,  
    Avg_cost DECIMAL(15,2) DEFAULT 0.00,  
    Sale_price DECIMAL(15,2) NOT NULL,  
    Stock_quantity DECIMAL(15,2) DEFAULT 0.00  
);
```

```
CREATE TABLE documents (  
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    Company_id UUID REFERENCES companies(id),  
    Partner_id UUID REFERENCES business_partners(id),  
    Doc_type VARCHAR(20) CHECK (doc_type IN ('PURCHASE_INVOICE',  
    'SALES_ORDER', 'PAYROLL')),  
    Status VARCHAR(20) DEFAULT 'DRAFT',  
    Total_amount DECIMAL(15,2) NOT NULL,  
    Xml_data JSONB,  
    Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE document_items (  
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    Doc_id UUID REFERENCES documents(id) ON DELETE CASCADE,  
    Product_id UUID REFERENCES products(id),  
    Quantity DECIMAL(15,2) NOT NULL,  
    Unit_price DECIMAL(15,2) NOT NULL,  
    Total_price DECIMAL(15,2) NOT NULL  
);
```

-- 4. RECURSOS HUMANOS

```
CREATE TABLE employees (  
    Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    Partner_id UUID REFERENCES business_partners(id),  
    Department VARCHAR(50),  
    Hire_date DATE NOT NULL,  
    Base_salary DECIMAL(15,2) NOT NULL,  
    Benefits_total DECIMAL(15,2) DEFAULT 0.00,  
    Tax_load_percent DECIMAL(5,2) DEFAULT 0.00  
);
```

-- 5. SEGURANÇA (RBAC)

```
CREATE TABLE roles (  
    Id SERIAL PRIMARY KEY,  
    Name VARCHAR(50) UNIQUE NOT NULL  
);
```

```
CREATE TABLE users (  

```

```
Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
Username VARCHAR(50) UNIQUE NOT NULL,
Password_hash TEXT NOT NULL,
Role_id INTEGER REFERENCES roles(id)
);
` ``
```

Fase 2: Lógica de Backend (Python + FastAPI)

Este código unifica as regras de negócio de **Compra, Venda e RH**.

```
` `` python

From fastapi import FastAPI, HTTPException
From pydantic import BaseModel
From typing import List, Optional
From datetime import datetime
Import uuid

App = FastAPI(title="Nexus ERP – API")

# --- MODELOS ---

Class Item(BaseModel):
    Product_id: str
    Quantity: float
    Unit_price: float

Class PurchaseInput(BaseModel):
    Vendor_id: str
```

```
Invoice_no: str  
Items: List[Item]  
Tax: float
```

```
Class SaleInput(BaseModel):
```

```
    Customer_id: str  
    Items: List[Item]
```

```
Class EmployeeInput(BaseModel):
```

```
    Partner_id: str  
    Base_salary: float  
    Tax_rate: float
```

```
# --- MÓDULO DE COMPRA (Praticidade + Rigor) ---
```

```
@app.post("/purchase/process")
```

```
Async def process_purchase(data: PurchaseInput):
```

```
    Total = sum(i.quantity * i.unit_price for i in data.items) + data.tax
```

```
    # Lógica Contábil Automática (SAP Style)
```

```
    Journal = {
```

```
        "debit": "ESTOQUE", "credit": "CONTAS_A_PAGAR", "amount": total
```

```
    }
```

```
    Return {"status": "Nota Lançada", "total": total, "contabilizacao": journal}
```

```
# --- MÓDULO DE VENDA (Rentabilidade) ---
```

```
@app.post("/sales/order")
```

```
Async def create_sale(data: SaleInput):
```

```
    Order_total = sum(i.quantity * i.unit_price for i in data.items)
```

```
    # Proteção de Margem e Crédito (Nexus Intelligence)
```

```
If order_total > 5000: # Simulação de limite
    Raise HTTPException(status_code=403, detail="Limite de crédito excedido.")
```

```
Return {"status": "Venda Aprovada", "total": order_total}
```

```
# --- MÓDULO DE RH (Custo Real) ---
```

```
@app.post("/hr/onboard")
```

```
Async def onboard(emp: EmployeeInput):
```

```
    # Cálculo de custo real (Custo Brasil simulação)
```

```
    Provision = emp.base_salary * (1/12 * 1.33 + 1/12)
```

```
    Total_cost = emp.base_salary + (emp.base_salary * emp.tax_rate) + provision
```

```
    Return {"status": "Admitido", "custo_mensal_real": round(total_cost, 2)}
```

```
    ``
```

```
### Fase 3: Inteligência e Automação (IA + Analytics)
```

```
O código para extrair inteligência e gerar o Dashboard de 100%.
```

```
#### **A. Lógica de IA para entrada de dados:**
```

```
```python
```

```
Def ai_invoice_ingestion(xml_json_data):
```

```
 # Simula a IA transformando XML em dados de sistema
```

```
 Return {
```

```
 "vendor_id": xml_json_data['cnpj'],
```

```
 "total": xml_json_data['valor_total'],
```

```
 "items": xml_json_data['itens'],
```

```
 "action": "AUTO_POSTING" # Lança sem digitação se bater com o pedido
```

```
 }
```

```

B. Query do Dashboard de Rentabilidade (Saída de Dados):

```sql

-- Executar para ver a saúde dos 100% da empresa

WITH Financeiro AS (

SELECT

(SELECT SUM(total\_amount) FROM documents WHERE  
doc\_type='SALES\_ORDER') as receita,

(SELECT SUM(total\_amount) FROM documents WHERE  
doc\_type='PURCHASE\_INVOICE') as custo\_compras,

(SELECT SUM(base\_salary \* (1 + tax\_load\_percent/100)) FROM employees) as  
custo\_rh

)

SELECT

Receita,

Custo\_compras,

Custo\_rh,

(receita – custo\_compras – custo\_rh) as lucro\_liquido,

ROUND(((receita – custo\_compras – custo\_rh) / receita) \* 100, 2) as  
margem\_percent

FROM Financeiro;

```

Fase 4: Ordem de Implementação Sugerida

1. **Semana 1:** Instalar PostgreSQL e rodar o script da **Fase 1**.
2. **Semana 2:** Configurar o ambiente Python e rodar o backend da **Fase 2**.

3. **Semana 3:** Criar as telas de interface (React/Next.js) conectando nas APIs criadas.
4. **Semana 4:** Implementar a **IA** de leitura de notas para eliminar a burocracia de entrada.

Parabéns! Você tem agora o esqueleto técnico completo do **Nexus ERP**. Se precisar de ajuda para codificar um endpoint específico ou integrar com um banco de dados real, estou aqui!